YOUR UVM REPO · NVME_REL

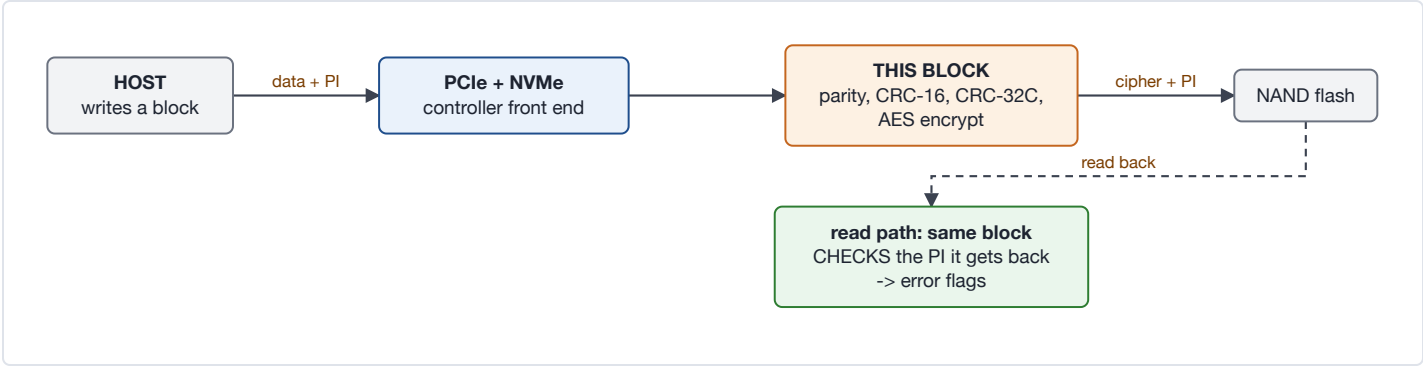
NVMe Reliability Block — AES, CRC-16, CRC-32C, Parity

What every file does · where it sits in an SSD controller · the algorithms from the specs with worked rounds · the NVMe Protection Information format · design and UVM flow step by step · 10 constraints · 10 assertions · results

1. What this block does, and why an SSD needs it

An SSD stores your data in NAND flash. On the way in and out, the data passes through buses, buffers and memories inside the controller — and any of them can flip a bit. A bit that silently flips is the worst kind of bug: the drive returns wrong data and nobody notices. So the controller **protects** every block of data in four ways:

Protection	What it catches	Defined by
Parity (1 bit per byte)	a single flipped bit in an internal buffer	the controller design (not in NVMe)
CRC-16 Guard	errors anywhere between the host and the media — this is NVMe **end-to-end data protection**	NVMe NVM Command Set spec (16b Guard Protection Information, T10-DIF CRC)
CRC-32C Guard	the same, much stronger (NVMe's 32b Guard format)	NVMe NVM Command Set spec (32b Guard Protection Information)
AES-128 encryption	someone reading the NAND chips directly (data at rest)	FIPS-197 (AES); self-encrypting drives use AES in XTS mode (IEEE 1619)



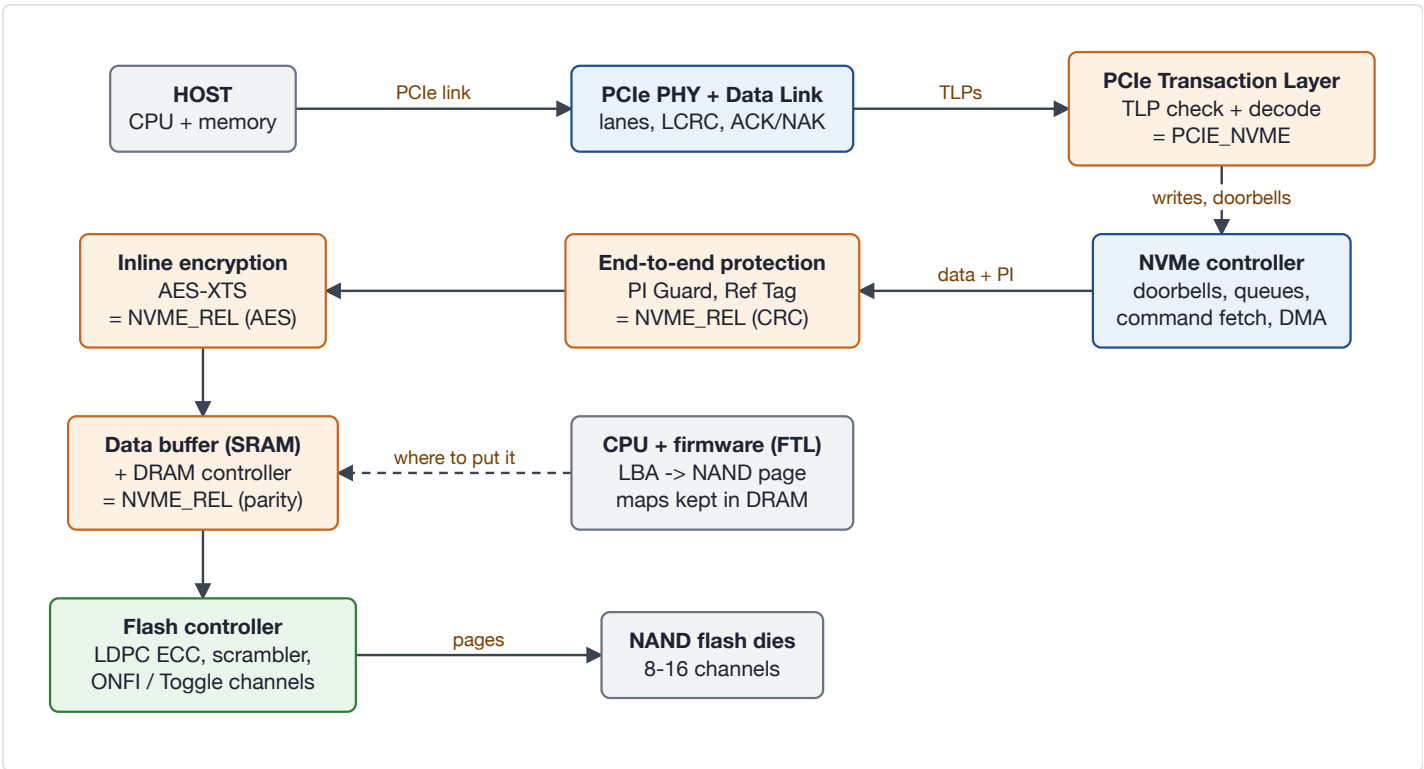
Two paths, one block:

- **Write path (`mode = 0`):** the block comes from the host → encrypt it with AES, and **generate** its protection: CRC-16 Guard, CRC-32C Guard, byte parity, App Tag, Ref Tag.
- **Read path (`mode = 1`):** the block comes back with the protection it was stored with → recompute everything and **check**: a mismatch raises an error flag (`guard16_err`, `guard32_err`, `reftag_err`, `parity_err`).

Simplification (on purpose, so it stays readable): a real logical block is 512 bytes or 4 KB. Here a block is **16 bytes** — exactly one AES block — so every number in this guide can be followed by hand. Every algorithm works the same way on longer blocks; only the loop gets longer.

2. Where this IP sits inside an SSD controller

An SSD controller is one chip with a **front end** (talks to the computer), a **middle** (moves and protects data, runs the firmware), and a **back end** (talks to the NAND flash). A write follows the arrows from the host down to the NAND; a read travels the same path backwards.



Part of the controller	What it does	Which of your blocks lives here
PCIe PHY + Data Link Layer	turns bits on the lanes into reliable packets: encoding, LCRC on every TLP, ACK/NAK replay	— (assumed done before your block)
PCIe Transaction Layer	checks each TLP (type, length, 4 KB rule), decodes the address against the BARs, reports Unsupported Request / Malformed TLP	**PCIE_NVME** — exactly this job
NVMe controller (front end)	doorbell registers, submission/completion queues, fetches commands, runs the **DMA** that moves data between host memory and the drive	**PCIE_NVME** hands it doorbell writes and data-window writes
End-to-end data protection	generates or checks the **Protection Information** (CRC Guard, App Tag, Ref Tag) at the host interface, and checks it again just before the media — so corruption anywhere inside the drive is caught	**NVME_REL** — CRC-16 / CRC-32C Guard, Ref Tag, App Tag escape
Inline encryption engine	encrypts every block on the way to NAND and decrypts on the way back (self-encrypting drive); real drives use **AES-XTS** with the LBA as the "tweak" so equal data at different LBAs looks different	**NVME_REL** — the AES-128 core (one block, no XTS tweak, to keep it simple)
Data buffer + DRAM (memory) controller	SRAM staging buffers and the DRAM that holds data in flight and the firmware's mapping tables; protected by **parity** (SRAM) or **ECC** (DRAM) because a bit flip here would silently corrupt user data or the map	**NVME_REL** — the byte parity
CPU cores + firmware (FTL)	the Flash Translation Layer decides which NAND page holds each LBA, does wear levelling and garbage collection	— (software)
Flash controller (back end)	strong **ECC** (LDPC or BCH) per NAND page, a **scrambler** so data looks random to the cells, read-retry, and the channel interface to the NAND dies	— (not in these blocks; the next level of protection)
NAND flash	the storage cells, many dies on several channels	—

Why so many layers of protection? Each one guards a different stretch of the path, like checkpoints on a road:

Protection	Guards	Strength
LCRC (PCIe Data Link)	the PCIe wire only	re-sends a bad packet
PI Guard CRC (NVME_REL)	host → whole controller → media, **end to end**	detects; the command fails instead of returning wrong data
Parity / DRAM ECC (NVME_REL parity)	one memory inside the controller	parity detects 1 bit; ECC corrects 1, detects 2
LDPC / BCH ECC (flash controller)	the NAND cells, which really do wear out and flip bits	corrects many bits per page
AES-XTS (NVME_REL AES)	not errors — **privacy** of the stored data	—

One write, from host to NAND, in order: host rings the SQ doorbell (**PCIE_NVME** decodes the TLP → doorbell) → NVMe controller fetches the command and DMAs the data in (more TLPs through **PCIE_NVME**) → **PI** is checked/generated (**NVME_REL** CRC) → data parked in the buffer (**parity**) → **encrypted** (**NVME_REL** AES) → firmware picks a NAND page → flash controller adds LDPC ECC and scrambles → programmed into NAND. A **read** runs the same path backwards: LDPC corrects the page → decrypt → PI checked again → DMA to the host.

3. The packet: a data block and its Protection Information (as in the NVMe spec)

NVMe end-to-end data protection attaches **Protection Information (PI)** to every logical block. The NVMe NVM Command Set specification defines several PI formats; the two this block produces:

16b Guard PI — 8 bytes per block (the classic T10-DIF format):

Bytes	Field	Contents in this design
0–1	**Guard**	CRC-16 (T10-DIF polynomial 8BB7h) over the block's data
2–3	**Application Tag**	any value the host chooses; **FFFFh disables the checks** for that block (Type 1)
4–7	**Reference Tag**	Type 1: the **lower 32 bits of the LBA** — proves the block is the one you asked for, not a neighbour

32b Guard PI — 16 bytes per block (NVMe's newer, stronger format):

Bytes	Field	Contents
0–3	**Guard**	CRC-32C (Castagnoli polynomial 1EDC6F41h) over the data
4–5	**Application Tag**	as above
6–15	**Storage and Reference Space** (80 bits)	holds the Reference Tag (and optional storage tag)

The packet this DUT sees (one transfer on its input, one on its output):

Field	Bits	Direction	Meaning
`in_mode`	1	in	0 = write path, 1 = read path
`in_data`	128	in	the 16-byte block; **byte 0 =** `in_data[127:120]` (first on the wire)
`in_key`	128	in	AES-128 key
`in_lba`	32	in	logical block address (lower 32 bits)
`in_app_tag`	16	in	Application Tag
`in_guard16`, `in_guard32`, `in_ref_tag`, `in_parity`	16, 32, 32, 16	in	read path only: the PI and parity stored with the block
`out_cipher`	128	out	AES-128 ciphertext of the block
`out_guard16`, `out_guard32`, `out_parity`	16, 32, 16	out	freshly computed protection
`out_app_tag`, `out_ref_tag`	16, 32	out	App Tag passed through; Ref Tag = LBA
`out_*_err`	1 each	out	read path: which check failed

Type 1 rules from the spec that the design implements: the Ref Tag must equal the low 32 bits of the LBA; an App Tag of **FFFFh** turns off the Guard and Ref Tag checks for that block (used for blocks that were never written with PI). Parity is internal, so it is always checked.

4. The algorithms — why, where the data comes from, how, with real numbers

All examples use the same block, the AES test vector from FIPS-197 Appendix C.1: **data =** `00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff`, **key =** `00 01 02 … 0f`.

4.1 Parity — the simplest check

Why: one flipped bit in a buffer changes the number of 1s from even to odd. Counting is enough to notice.

How: even parity of a byte = XOR of its 8 bits (1 if the byte has an odd number of 1s). One parity bit per byte → 16 bits for the block; bit 15 belongs to byte 0.

Byte	Binary	Number of 1s	Parity bit
`11`	0001 0001	2	0
`13`	0001 0011	3	1

Every byte of our example block has an even number of 1s, so its parity is `0000`. **Limit:** two flipped bits in the same byte cancel out — that's why the CRCs exist.

4.2 CRC-16 Guard (T10-DIF) — catching bursts of errors

Why: a CRC treats the whole block as one huge binary number and divides it by a fixed "generator" polynomial; the **remainder** is the CRC. Any error that is not a multiple of the polynomial changes the remainder — that covers every single-bit and double-bit error and every burst up to 16 bits long. The receiver recomputes the remainder and compares.

Where the data comes from: the block's bytes, byte 0 first, most significant bit first (that's the T10-DIF convention: no bit reflection, start value 0000h, no final XOR). Polynomial **8BB7h** = $x^{16} + x^{15} + x^{11} + x^9 + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$.

How (the shift register): for each byte, XOR it into the top of the 16-bit register; then 8 times: shift left by one, and if the bit that fell out was 1, XOR in 8BB7h. "XOR the polynomial when the top bit is 1" **is** long division in binary, one bit at a time.

```
c = c ^ {byte, 8'h00};                              // bring the next byte into the top
for (int b = 0; b < 8; b++)
  c = c[15] ? ((c << 1) ^ 16'h8BB7) : (c << 1);     // one step of long division
```

Worked example — the first two bytes:

After	CRC register	Why
byte 0 = `00`	`0000`	0 XOR anything-from-0 stays 0
byte 1 = `11` : load	`1100`	`11` goes into the top byte
step 1	`2200`	top bit 0 → just shift
step 2	`4400`	top bit 0 → shift

After	CRC register	Why
step 3	`8800`	top bit 0 → shift
step 4	`9BB7`	top bit 1 → shift (`1000`) then XOR `8BB7`
step 5	`BCD9`	top bit 1 → shift, XOR
step 6	`F205`	top bit 1 → shift, XOR
step 7	`6FBD`	top bit 1 → shift, XOR
step 8	`DF7A`	top bit 0 → shift
byte 2 = `22`	`B36A`	same 8 steps
… all 16 bytes	**`B1B2`**	this is the Guard for the example block

How we know the code is right: the published check value of CRC-16/T10-DIF for the ASCII text `123456789` is `D0DB` — the Python reference reproduces it, and the DUT matches the reference.

4.3 CRC-32C Guard — the same idea, stronger

Why a second CRC: 16 bits means 1 in 65 536 random corruptions slip through; 32 bits makes it 1 in 4 billion. NVMe's 32b Guard format uses **CRC-32C (Castagnoli)**, chosen because it detects more error patterns than the older Ethernet CRC-32 and many CPUs compute it in one instruction.

What's different from CRC-16: the bits are processed **least significant first** ("reflected"), so the register shifts **right** and the polynomial is written reversed: 1EDC6F41h becomes **82F63B78h**. It starts at FFFFFFFFh (so leading zero bytes still change the CRC) and the result is inverted at the end.

```
c = c ^ {24'h0, byte};                              // the byte enters at the BOTTOM
for (int b = 0; b < 8; b++)
  c = c[0] ? ((c >> 1) ^ 32'h82F6_3B78) : (c >> 1);   // shift right, XOR when a 1 falls out
...
return c ^ 32'hFFFF_FFFF;                           // final inversion
```

After	Register (before the final XOR)
start	`FFFFFFFF`
byte 0 = `00`	`AD82ACAE`
byte 1 = `11`	`ECABCC41`
… all 16 bytes, then XOR FFFFFFFFh	**`48DFE982`** = the 32b Guard

Check value for `123456789` : `E3069283` — reproduced by the reference.

4.4 AES-128 — encryption, round by round

Why: the NAND chips can be unsoldered and read. AES scrambles every block with a secret key so the stored bits are useless without it. Encryption has to be **reversible** (with the key) but look **random** (without it) — that's why AES mixes the data in four different ways, ten times.

Where the data comes from: the 16 data bytes are arranged in a 4 × 4 grid called the **state**, filled **column by column**: bytes 0–3 are column 0, bytes 4–7 column 1, and so on. The 16 key bytes become the first **round key**; the **key schedule** makes 10 more round keys from it, one per round.

```
          col0 col1 col2 col3
row 0:     00   44   88   cc        <- the example block 00 11 22 ... ff
row 1:     11   55   99   dd
row 2:     22   66   aa   ee
row 3:     33   77   bb   ff
```

The four steps of a round, and why each exists:

Step	What it does	Why
SubBytes	every byte is replaced through a fixed table, the **S-box**	the only **non-linear** step — without it AES would be solvable with simple algebra
ShiftRows	row r is rotated left by r places	spreads each column's bytes over different columns
MixColumns	each column is multiplied by a fixed matrix in GF(2^8)	every output byte depends on all 4 input bytes of the column — one changed bit spreads everywhere
AddRoundKey	XOR with this round's key	the only step that uses the secret

Round 0 is only AddRoundKey: `00112233…eeff` XOR `00010203…0e0f` = **`00102030 40506070 8090a0b0 c0d0e0f0`**.

Round 1 (every value matches FIPS-197 Appendix C.1):

Step	State after it	Where it came from
start	`00102030 40506070 8090a0b0 c0d0e0f0`	round 0
SubBytes	`63cab704 0953d051 cd60e0e7 ba70e18c`	e.g. S-box[`00`] = `63`, S-box[`10`] = `ca`
ShiftRows	`6353e08c 0960e104 cd70b751 bacad0e7`	row 1 moved left by 1, row 2 by 2, row 3 by 3
MixColumns	`5f726415 57f5bc92 f7be3b29 1db9f91a`	see the column example below
AddRoundKey	XOR with round key 1 `d6aa74fd d2af72fa daa678f1 d6ab76fe`	from the key schedule (below)

Round 2:

Step	State
start	`89d810e8 855ace68 2d1843d8 cb128fe4`
SubBytes	`a761ca9b 97be8b45 d8ad1a61 1fc97369`
ShiftRows	`a7be1a69 97ad739b d8c9ca45 1f618b61`
MixColumns	`ff879684 31d86a51 645151fa 773ad009`
AddRoundKey with `b692cf0b 643dbdf1 be9bc500 6830b3fe`	→ round 3 starts with `4915598f 55e5d7a0 daca94fa 1f0a63f7`

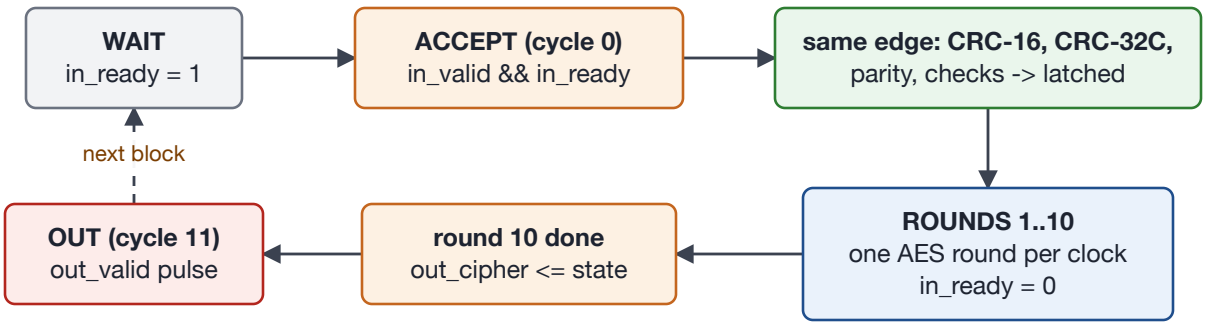
Rounds 3 to 9 repeat this; **round 10 skips MixColumns**; the result is **`69c4e0d8 6a7b0430 d8cdb780 70b4c55a`** — exactly the ciphertext published in FIPS-197.

MixColumns, one byte by hand (round 1, column 0 = `63 53 e0 8c`): the first output byte is `2·63 ⊕ 3·53 ⊕ e0 ⊕ 8c`. "Multiply by 2" in GF(2^8) is a left shift, plus XOR `1b` if a bit fell out (`xtime`): `2·63 = c6`. "Multiply by 3" = `2·x ⊕ x`: `3·53 = a6 ⊕ 53 = f5`. Then `c6 ⊕ f5 ⊕ e0 ⊕ 8c = 5f` ✓ — the first byte of the MixColumns row above. The `1b` comes from the AES polynomial $x^8 + x^4 + x^3 + x + 1$, which keeps every result inside one byte.

Where an S-box value comes from: S-box[a] = the multiplicative **inverse** of a in GF(2^8), then a fixed bit-mix (affine transform) with `63`. Example: inverse of `10` is `74`; affine(`74`) = `ca` ✓. The DUT stores the 256 results as a table; the testbench's reference model **computes them from this math** — two different ways of getting the same table, which is why the comparison means something.

Key schedule, first word of round key 1: take the last key word `0c0d0e0f`, rotate it (`0d0e0f0c`), push each byte through the S-box (`d7ab76fe`), XOR the round constant `01` into the first byte (`d6ab76fe`), and XOR with the first key word `00010203` → **`d6aa74fd`** ✓. The other three words are each the XOR of the word before and the matching old word. The round constants `01 02 04 08 10 20 40 80 1b 36` stop the round keys from repeating.

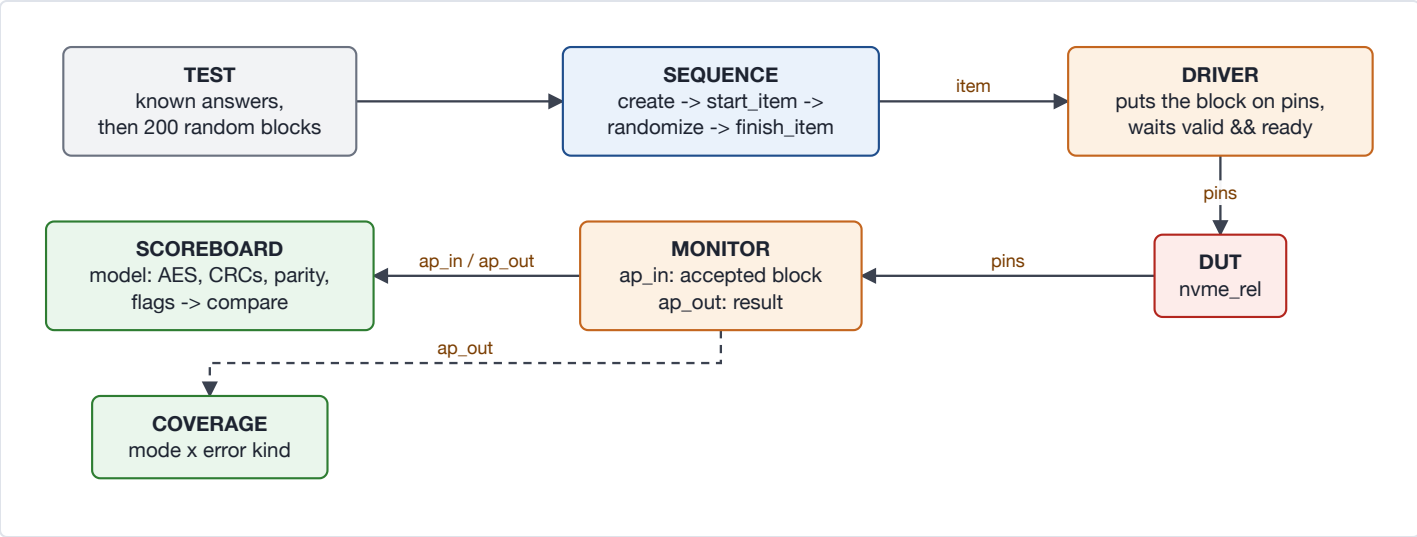
5. The design, step by step



Cycle	What happens inside `nvme_rel.sv`
0	`in_valid && in_ready`: state ← data ⊕ key (round 0); CRC-16, CRC-32C, parity and all checks are computed **combinationally** and stored; `busy` = 1 → `in_ready` = 0
1 … 9	one full AES round per clock: `state ← aes_round(state, next_key(rkey))`; the round key is also advanced one step
10	last round (no MixColumns); `out_cipher` ← result; `out_valid` ← 1; `busy` ← 0
11	`out_valid` is seen high for one cycle together with all outputs; `in_ready` is 1 again

Why one round per clock: a full AES in one clock would be a very long chain of logic (a slow chip). Doing one round per clock re-uses the same round logic ten times — slower per block, much smaller. Real controllers pipeline 10 copies to get one block per clock.

6. The UVM testbench, step by step



1. `tb_top` makes the clock and reset, connects the DUT to `nvme_rel_if`, puts the interface in `config_db`, calls `run_test`.
2. The **test** builds the env and starts `nvme_rel_kat_seq` (the FIPS block, once on the write path and once on the read path), then `nvme_rel_sequences` (200 random blocks).
3. Each **item** is randomized under the 10 constraints; `post_randomize()` fills the data pattern and, for a read-path block, the **correct** PI — then flips one bit of a field if an error knob is set.
4. The **driver** drives all fields and holds `in_valid` until it sees `in_valid && in_ready` at the same clock edge.
5. The **monitor** writes the accepted block to `ap_in` and, 11 cycles later, the result to `ap_out`.
6. The **scoreboard** computes the expected result with `nvme_rel_model` (AES, CRC-16, CRC-32C, parity, expected flags), queues it, and compares when the result arrives. For the FIPS block it also checks the model against the published answers.
7. **Coverage** (fed from the monitor's ap_out) samples every result: write/read × which error (none, Guard16, Guard32, Ref Tag, parity) × App Tag escape.
8. The 10 **assertions** in `nvme_rel_if.sv` check the handshake and timing every clock.
9. `nvme_rel_err_test` replaces every random item with `nvme_rel_err_item` (factory override) → every block is a read with exactly one wrong field.

7. Every file and what it does

File	What it does
`nvme_rel.sv`	**DUT**: AES-128 (one round per clock), CRC-16 T10-DIF, CRC-32C, byte parity, PI generation and checking
`nvme_rel_if.sv`	Interface: all wires, driver and monitor clocking blocks, **the 10 assertions**
`testbench.sv`	Top: clock, reset, DUT, interface → `config_db`, `run_test`, waveform dump
`nvme_rel_pkg.sv`	Package: includes every class; FIPS-197 known answers (key, plaintext, ciphertext, CRCs)

File	What it does
`nvme_rel_model.sv`	**Reference model**: the same algorithms written a second way — S-box computed from GF(2^8) math, table-driven CRCs, byte-array AES
`nvme_rel_item.sv`	The transaction: mode, data, key, LBA, App Tag, error knobs; `post_randomize()` builds the PI; constraints 1–5, 7, 8, 10
`nvme_rel_err_item.sv`	Error block: replaces `c_good` (exactly one error) and `c_mode` (read path) — constraints 9 and 6
`nvme_rel_sequences.sv`	Random sequence (200 blocks) + known-answer sequence (FIPS block on both paths)
`nvme_rel_sequencer.sv`	Sequencer
`nvme_rel_driver.sv`	Drives one block, handshake on `in_valid && in_ready`
`nvme_rel_monitor.sv`	`ap_in` (accepted inputs), `ap_out` (results)
`nvme_rel_agent_cfg.sv`	Active/passive, coverage on/off
`nvme_rel_agent.sv`	Builds monitor (+ driver, sequencer when active), connects them
`nvme_rel_scoreboard.sv`	`_imp_decl(_in/_out)`; predicts with the model, compares 10 fields, known-answer self-check
`nvme_rel_coverage.sv`	Covergroup: mode × error kind, App Tag escape
`nvme_rel_env.sv`	Agent + scoreboard + coverage, connections
`nvme_rel_tests.sv`	`nvme_rel_tests` (known answers + random), `nvme_rel_err_test` (factory override)
`run.sh`	Compile once, run both tests, logs and waves into `results/<test>/`

8. The 10 constraints

#	Name	Rule	Why
1	`c_mode`	write / read 50 / 50	both paths equally
2	`c_pattern`	random data 70%, all-0 / all-1 / `A5` pattern 30%	special patterns are where CRC and AES bugs show
3	`c_err_mode`	an error knob ⇒ read path	errors only make sense when checking
4	`c_lba`	LBAs near 0 and near FFFFFFFFh weighted up	edges of the Ref Tag
5	`c_app`	App Tag FFFFh 10%	exercises the spec's "checks disabled" escape
6	`c_mode` (err item)	error blocks are read-path blocks	replaces constraint 1 in the child
7	`c_idle`	1–3 idle cycles between blocks	gaps on the interface
8	`c_good`	no error knob set	normal traffic is clean
9	`c_good` (err item)	exactly one error knob	one fault at a time → a clear expected flag
10	`c_key`	key ≠ 0	realistic keys

9. The 10 assertions

#	Name	Rule
1	`a_hold`	input stable while `in_valid && !in_ready`
2	`a_latency`	accept → `out_valid` exactly 11 cycles later
3	`a_pulse`	`out_valid` lasts one cycle
4	`a_busy`	after an accept, `in_ready` drops
5	`a_ready`	`in_ready` is high again when the result comes out
6	`a_cause`	no `out_valid` without an accept 11 cycles before
7	`a_no_x`	no X on the result
8	`a_wr_clean`	the write path never raises an error flag
9	`a_escape`	App Tag FFFFh ⇒ no Guard / Ref Tag error (spec)
10	`a_reset`	after reset: `out_valid` = 0, `in_ready` = 1

10. Results and how to run

Test	Result
`nvme_rel_tests`	202 blocks pass, 0 fail; the FIPS block matched the published AES ciphertext and both CRCs (2 known-answer checks); 0 assertion failures
`nvme_rel_err_test`	202 blocks pass; **184** planted errors flagged exactly as predicted (the others had App Tag FFFFh, which disables the PI checks — correct per the spec)
Bug injected (wrong CRC polynomial + one MixColumns term)	**202 of 202** blocks failed — e.g. `cipher 2d17… expected 69c4…` , `guard16 34b0 expected b1b2`

Run: `./run.sh` (both tests) or `./run.sh nvme_rel_err_test` . Logs and `dump.vcd` go to `results/<test>/` ; open the waves with `surfer results/nvme_rel_tests/dump.vcd` . Tools: `../ADDER/setup_tools.sh` once.

Two simulator lessons met while building it: (1) Verilator treats `dist` as a hard choice, so two `dist` rules linked by an implication can fail where VCS/Questa wouldn't — the constraints are written so no two `dist` rules are linked. (2) The known-answer items are built with `new()` , not the factory, so the error test's override can never change them.